

Reviewer Pack — Hilbert Function Growth of Monomial Ideals Bounds ACC^0 Circ...

Entry 8ab2a7e6d85a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 03:25:25 UTC

Hilbert Function Growth of Monomial Ideals Bounds ACC^0 Circuit Size

Entry ID: 8ab2a7e6d85a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 03:25:25 UTC

1. Conjecture

Field A (mathematical branch): Commutative Algebra **Field B** (complexity object): ACC^0 Circuit Size

Statement:

For any CNF formula with n variables, the Hilbert function of the monomial ideal generated by its clauses satisfies $H(n) = \Omega(\log n)$ iff the ACC^0 circuit size is $\Omega(n)$.

Rationale (proposer's reasoning):

The Hilbert function captures the asymptotic growth of graded components in monomial ideals, which may encode structural complexity that correlates with circuit size. This bridges algebraic geometry with circuit lower bounds via ideal generation.

Taxonomy category: $ACC_LB_via_WILLIAMS$ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ac172fc28301d427

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=2.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for j in range(i)):
                clauses.append(tuple(sorted(clause)))
        return set(clauses)

    def monomial_to_index(monomial):
        return sum(2 ** (abs(x) - 1) for x in monomial if x > 0)

    def hilbert_function(I, n):
        H = [0] * (n + 1)
        for k in range(n + 1):
            count = 0
            for clause in I:
                if all(abs(x) <= k for x in clause):
                    count += 1
            H[k] = count
        return H

    def acc0_circuit_size_lower_bound(n):
        # A simple lower bound for ACC^0 circuit size (e.g.,  $n * \log n$ )
        return n * math.log(n, 2)

    n = random.randint(5, 40)
    I = generate_3cnf(n)
    H = hilbert_function(I, n)
    acc0_bound = acc0_circuit_size_lower_bound(n)

    metric_value = H[n] / (n * math.log(n, 2))
    conjecture_holds = (H[n] >= 1) == (acc0_bound >= 1)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Hilbert Function Growth",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
```

```

        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d65c6c07.py", line 68, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d65c6c07.py", line 47, in run_trial
    I = generate_3cnf(n)
        ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d65c6c07.py", line 25, in generate_3cnf
    if all(clause[i] != -clause[j] for j in range(i)):
            ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples. | next: Fix the NameError in generate_3cnf() by defining 'i' in the loop condition and re-run trials

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	54632
2	preregistration	ollama_remote	qwen3:8b	0	0	32442
3	novelty	ollama_remote	qwen3:8b	0	0	27465
4	novelty	ollama_remote	qwen3:8b	0	0	22342
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14552
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16977
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9823
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16387
9	judge	ollama_remote	qwen3:8b	0	0	117112

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 311732 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8ab2a7e6d85a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8ab2a7e6d85a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8ab2a7e6d85a.tar.gz` (if generated)